

Provably: Fair Outcomes and Provably Identical Rules

Slid Phi Labs -- 26 September 2026

1. What "provably fair" proves today -- and what it doesn't

Every provably-fair casino in operation works the same way. Before you bet, the house publishes a commitment -- usually SHA-256 of a secret server seed. You supply a client seed. After the bet, the house reveals the server seed, and you can re-run the math yourself: the outcome was decided before your bet, so the house couldn't have rigged that particular roll against you.

That half works. It is genuinely provable.

Here is the half that isn't. You verified the **outcome**. You did not verify the **rules**. The program that ran on the house's server -- the one that turns seeds into a roll, a card, a flip -- is a black box. The house can show one ruleset to an auditor and run a different one in production, and no amount of seed math will catch it. Players verify the dice; nobody verifies the table.

Provably closes that gap. It makes the rules themselves provable: the exact same game program, running identically, on every machine that runs it -- or the program is refused outright.

2. The protocol

Provably has two halves, and both have to hold for a game to be called fair.

Half 1 -- commit-reveal (the outcome is provable). Standard practice, unchanged:

1. The house publishes ``commit = SHA-256(server_seed)`` before any bet.
2. The player provides a client seed (or accepts a random one, recorded).
3. After the round, the house reveals ``server_seed``.
4. The player checks that ``SHA-256(server_seed)`` matches the published commitment. If it doesn't, the house cheated and the proof is public.

Half 2 -- exactness (the rules are provable). This is the new part:

1. Every game is written once, in one source file (the reference games live in ``games/`` in this repository).
2. That source compiles to every target seat -- Python, JavaScript, TypeScript, C, C++ -- and every seat must produce ****byte-identical**

output** for the same inputs. Not "equivalent". Byte-identical.

3. If any seat disagrees, or if a program can't be compiled exactly, the compiler **refuses** the program. There is no "close enough" mode.

4. The gate writes a **source-hash receipt**: SHA-256 of the exact source that was gated, the pass/fail verdict, and per-seat emit/run status.

The consequence: an auditor reads one short program, gates it, and knows that the program the house runs in production is the same program -- because any change to the source changes the source hash, and any behavior change that differs across seats fails the gate. One ruleset for the audit is the only ruleset there is.

Why byte-identical, not just equivalent

Floating point, integer overflow, string encoding, and modulo on negatives all differ across languages and platforms. A game that is "equivalent" everywhere is a game whose edge cases someone gets to choose. Byte-identical output removes the choice. The reference games use only integer arithmetic with explicit modulus (the Park-Miller step, ``state = (48271 * mixed) mod 2147483647``), so there is no platform-dependent behavior to hide behind.

What the program contains -- and doesn't

The program is deterministic: seeds go in, the result comes out. There is no randomness inside the program. The seeds *are* the randomness, and they come from the commit-reveal half. This separation is deliberate: randomness you can audit (revealed seeds) stays outside; rules you can audit (the program) stay inside. Nothing about the outcome is left to the machine's mood.

3. The reference games

Two games ship in this repository, both gated 2026-09-26 on the five-seat matrix (py, js, ts, c, cpp). Both passed: every seat byte-identical.

Game 1 -- Dice (``games/provably-fair-dice.cuni``)

Mixes ``server_seed``, ``client_seed``, and ``round`` with a Park-Miller step, then takes the roll as ``(state % 6) + 1``. Also exposes a 0-99 roll.

Audited outputs (same seeds, every seat, 9 bytes total):

```
3
4
1
84
```

Source SHA-256:

```
`4627936386e0e69a875bdf473dc912167aa4449001fe8e7e96c55c52dbf6034d`
```

Stdout SHA-256:

```
`a79a72f0b7987fab1a4e54c85f617a723f5d2b25aa1d3b2f2fc3a2a470ba5f8b`
```

Game 2 -- Coin flip (`games/provably-fair-coin-flip.cuni`)

Same derivation shape as the dice game (seed mix + Park-Miller step); the flip is heads when the state is even, tails when odd. Also exposes the raw 0/1 bit for payout math.

Audited outputs (same seeds, every seat, 52 bytes total):

```
heads
tails
heads
tails
heads
heads
tails
tails
0
1
```

Source SHA-256:

```
`04783043adaf23b7880977bfaacc4324eb02d8d77bf41991f7dfd6037510c3d0`
```

Stdout SHA-256:

```
`da48a6ba509f0840da7040eaa51961e9506334e043601253c534437a091202b7`
```

Worked verification -- follow along (players and operators)

You need only Python 3 (or node, gcc -- any one seat) and the game's source.

1. **Get the source.** Download `games/provably-fair-coin-flip.cuni` from this repository, or from the house's published copy.
2. **Check the source hash.** Run `sha256sum` on the file. It must equal the source hash in the receipt (`receipts/provably-fair-coin-flip.receipt.json`) and the hash the house published with its game listing. If it doesn't match, stop -- that's a different game.
3. **Get the revealed seeds.** After your round, the house reveals its `server\_seed`. Check `SHA-256(server\_seed)` against the commitment it published before you bet.
4. **Re-run the round.** Plug the revealed server seed, your client seed, and the round number into the program's `flip` function (any seat -- the gate guarantees they all agree). The output must match what the house

reported for your round.

5. **Check the receipt.** The receipt in this repository records the gated verdict and per-seat results. An independent auditor can re-run `./gate/run-gate.sh`` and reproduce the same PASS.

If every step checks out, two facts hold: the outcome was fixed before your bet (commit-reveal), and the rules that produced it are the audited rules (exactness). Either half failing is a caught cheat.

4. Source-hash receipts

A receipt (`receipts/*.receipt.json``) is a small signed-by-math artifact:

- ``path`` -- which program was gated
- ``source_hash`` -- SHA-256 of the exact bytes that were gated
- ``exact` / `summary`` -- the verdict (``exactness: PASS (5 langs)``)
- ``langs` / `seats`` -- per-seat emit and run status

Receipts are how a house answers "prove you're running the audited game": publish the receipt alongside the source hash. A player hashes the source, compares, and knows. An auditor re-runs the gate and reproduces the verdict. The receipt is not trust -- it's a checkable claim.

5. What this is NOT

Plainly, so there's no confusion:

- **Not a casino.** Provably is a protocol and two reference games. It takes no bets, holds no funds, and operates no tables.
- **Not a license.** Nothing here grants or implies a gaming license in any jurisdiction. Operators are responsible for their own compliance.
- **No custody of funds.** There is no wallet, no escrow, no token. Money movement is entirely outside this protocol.
- **Not a randomness source.** The protocol consumes seeds; it does not generate them. Seed generation and commitment handling are the operator's job, and they must be done honestly -- the math only catches cheating, it doesn't prevent a dishonest setup.
- **Two games, not a platform.** Dice and coin flip demonstrate the pattern. Blackjack, roulette, slots -- each new game needs its own source, its own gate pass, its own receipt. The pattern scales; the games don't write themselves.

6. For operators: what you get

- **A defense against your own worst incident.** A rigged-rules scandal ends a casino. Exactness makes "we ran different code in production" technically checkable by anyone, which is the strongest form of "we didn't."
- **Audits that mean something.** An auditor gates one short program instead of reviewing your whole stack. The receipt is the audit artifact.
- **Player trust you can point at.** Every verification step above is something a player can actually do, tonight, with free tools.

Start with ``OPERATOR_ONEPAGER.md`` (one page), then this paper, then ``gate/run-gate.sh``.

*Reference implementation and gate by Slid Phi Labs, Cherry Hill, NJ.
Protocol: commit-reveal seeds + exactness law (same program, every seat, byte-identical or refused). License: AGPL-3.0, commercial licenses available.*